

Course Project: EE656
(Artificial Intelligence Machine Learning
Deep Learning & Its Application)

Self-Optimal Clustering Technique Using Optimized Threshold Function

TEAM MEMBERS:

AYUSH KUMAR (240239)
PRINCE MEENA (240796)
GAUTAM PRAJAPAT (240397)
ARUN CHAUHAN (240198)
AALOK (240011)

WHAT IS CLUSTERING ?

- Clustering is an unsupervised machine learning technique that groups objects such that similar objects fall within the same cluster and dissimilar objects are separated into different clusters.
- There are two types of clustering

Hard Clustering	Fuzzy Clustering
Each point belongs to exactly one cluster	Each point has a degree of membership across clusters
Rigid, crisp boundaries	Soft, overlapping boundaries
Example: K-Means	Example: Fuzzy C-Means (FCM)

- **Applications**

1. Image Segmentation — Grouping pixels by color/texture regions
2. Fraud Detection — Isolating anomalous transaction patterns
3. Bioinformatics — Clustering gene expression data
4. Pattern Recognition — Identifying structural groups in feature space

EXISTING METHODS AND THEIR LIMITATIONS

Method Comparison Table:

Method	Key Idea	Limitation
K-Means	Centroid-based partitioning	Assumes spherical clusters; requires k specified in advance
Fuzzy C-Means	Soft membership degrees	Sensitive to initial partitions; prone to local minima
Mountain Clustering	Grid-based center detection	Complexity grows exponentially with dimensions
DBSCAN	Density-based; handles arbitrary shapes	Requires careful tuning of ϵ and minPts parameters
IMC-1	Iterative cluster removal, fixed threshold ($\beta=1$)	Heuristic threshold; no data-driven adaptation
IMC-2	Multiplies threshold by heuristic factor $nk/m+1$	No mathematical justification; sometimes underperforms IMC-1



These gaps collectively motivate the need for a self-adaptive, mathematically grounded clustering approach → SOC

CLUSTER QUALITY METRICS

GSI — Global Silhouette Index (Range: -1 to 1 | Higher is Better)

- Measures how well each point fits its assigned cluster vs. the nearest neighboring cluster. Value close to 1 = excellent separation and compactness.

PI — Partition Index (Lower is Better)

- Ratio of within-cluster scatter to between-cluster distance — lower means clusters are tight and well-separated.

SI — Separation Index (Lower is Better)

- Uses minimum-distance separation between cluster centers. Penalizes partitions where clusters are too close.

DI — Dunn Index (Higher is Better)

- Ratio of minimum intercluster distance to maximum intracluster diameter. Rewards compact, well-separated clusters.

WHY SOC? – THE NEED FOR OPTIMIZATION

K-Means

Requires the number of clusters k to be specified manually before running. Assumes all clusters are spherical and equal in size — fails on real-world irregular data.

Fuzzy C-Means

Sensitive to how clusters are initialized — different starting points give different results. Often gets stuck in local minima and never finds the true partition.

IMC-1

Uses a fixed threshold $\beta=1$ for every cluster regardless of the data. A dense cluster and a sparse cluster get the same neighborhood radius — clearly wrong.

IMC-2

Tries to fix IMC-1 by multiplying the threshold by a ratio of cluster sizes ($n_k/m+1$). But this factor has no mathematical derivation — it was chosen by intuition. In our experiments, IMC-2 actually performed worse than IMC-1 on 2 out of 3 synthetic datasets.

The common gap across all methods:

- No method uses cluster quality as feedback to improve the threshold
- Thresholds are either fixed, assumed, or guessed

SOC ALGORITHM – STEP BY STEP

01.

Normalize data into unit hypercube

02.

Calculate threshold δm with initial $\beta m = 1$

03.

Compute potential using Gaussian mountain function

04.

Select highest-potential point as cluster center

05.

Assign points within δm distance to cluster

06.

Remove assigned points from dataset

07.

Repeat for next cluster

08.

Assign remaining points to nearest cluster center

SOC ALGORITHM – STEP BY STEP

09.

Compute GSI using
silhouette index for all
clusters

10.

Apply Lagrange interpolation
on (δ_m, S_m) pairs

11.

Set $S_t = 1$ (ideal) in polynomial
to find optimal threshold η

12.

Select root η giving
silhouette closest to 1

13.

Update $\beta_m = \eta / \delta_m$ for each
cluster

14.

Repeat for 10 iterations: track and
return the configuration with the
highest GSI.

LAGRANGE INTERPOLATION – THE CORE IDEA

- SOC uses Lagrange interpolation to find the optimal threshold η that maximizes the global silhouette index (GSI). Lagrange interpolation is a method to fit a unique polynomial curve through any set of data points — given M points, it gives exactly one polynomial of degree $M-1$.

PROCESS:

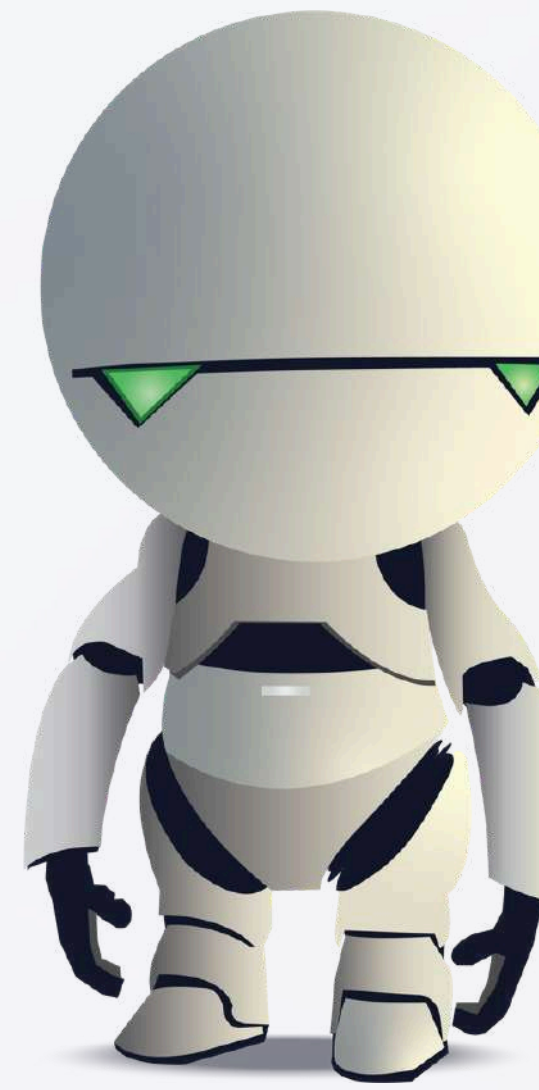
- A unique polynomial is constructed passing through all M pairs of (δ_m, S_m) — capturing the exact relationship between threshold and cluster quality
- For M clusters, this yields an $(M-1)$ th degree polynomial — one degree of freedom per cluster
- The polynomial is expressed as

$$S_t = \sum_{m=1}^M S_m l_m(\delta_t), \quad l_m(\delta_t) = \prod_{\substack{k=1 \\ k \neq m}}^M \frac{\delta_t - \delta_k}{\delta_m - \delta_k}$$

- The root η closest to $S_t = 1$ is selected, and the optimizing factor is updated as:
 $\beta_m = \eta / \delta_m$, for all $m = 1, 2, \dots, M$

$S_t=1$ means perfect silhouette — we find the threshold η that achieves this.

BENEFIT: This approach systematically finds the threshold that yields the best clustering quality — replacing heuristic trial-and-error with a mathematically grounded, data-driven solution.



MATHEMATICAL CONVERGENCE OF SOC

SOC's threshold δ_m is mathematically proven to converge — not just empirically observed to improve.

The Problem with $\beta = 1$:

The threshold in IMC-1 has no mechanism to improve — β stays at 1 forever. SOC introduces a β that is recomputed each iteration via Lagrange root-finding, and this iterative update is proven to converge conditionally.

The Convergence Function:

The threshold across iterations takes the form:

$$\delta_i = (\text{const.})^I \cdot f(\eta_{i-1})$$

And the Lagrange silhouette function is expressed as:

$$S_i = (\text{const.})^I \cdot f(\delta_i)$$

What "Conditional Convergence" means:

SOC runs for exactly 10 iterations. GSI does not improve smoothly — it oscillates up and down across iterations. The peak GSI in the 2-color example occurs at iteration 9, not 10. SOC handles this by tracking the best GSI seen across all iterations and returning that — not just the last result.

This oscillating but bounded behavior is called conditional convergence — the optimal value is reached, just not in a straight line.

THRESHOLD FUNCTIONS – SOC VS OTHERS

Threshold defines the boundary around a center: any point falling within this radius is pulled into the cluster, while points outside are left out. It acts like a net cast around your data:

Too small: The net misses points that belong together, leading to fragmentation (too many clusters).

Too large: The net grabs points from neighboring clusters, causing incorrect merges (too few clusters).

Method	Threshold Function	Modification
IMC-1	Base formula	None
IMC-max	min replaced by max	Heuristic
IMC-half	Factor 1/2 removed	Heuristic
IMC-2	External factor $nk/(nk+1)$	Heuristic
SOC	Base $\times \beta_m$	Data-driven

- IMC-1 — Never changes the threshold. β_m stays 1 forever. Simple but rigid.
- IMC-max / IMC-half — Someone tried tweaking the formula by swapping min→max or removing the 1/2. No logic behind it. Both perform worse than IMC-1 in most cases.
- IMC-2 — Multiplies threshold by a ratio of cluster sizes. Sounds reasonable but has no mathematical proof. Works sometimes, fails other times.
- SOC — Computes β_m by solving a polynomial equation. The threshold is earned from the data, not assumed.

SOC is the only one that asks the data what the threshold should be.

ITERATION BEHAVIOUR

- To understand how SOC improves over iterations, the paper tracked key values across all 10 runs for a simple two-color image (M=2).

Iter	δ_m		S_m		GSI	η	β_m	
	δ_1	δ_2	S_1	S_2			β_1	β_2
1	0.0326	0.0617	0.9093	0.7318	0.8206	0.0177	0.5429	0.2869
2	0.0177	0.0166	0.9170	0.7285	0.8228	0.0181	1.0226	1.0904
3	0.0334	0.0677	0.9093	0.7318	0.8206	0.0159	0.4760	0.2349
4	0.0155	0.0135	0.9170	0.7285	0.8228	0.0164	1.0581	1.2148
5	0.0344	0.0752	0.9093	0.7318	0.8206	0.0135	0.3924	0.1795
6	0.0128	0.0101	0.8484	0.7455	0.7970	0.0168	1.3125	1.6634
7	0.0427	0.1069	0.9154	0.7297	0.8226	0.0135	0.3162	0.1263
8	0.0103	0.0069	0.8484	0.7455	0.7970	0.0152	1.4758	2.2029
9	0.0481	0.1412	0.9251	0.7268	0.8260	0.0130	0.2703	0.0921
10	0.0088	0.0049	0.8484	0.7455	0.7970	0.0144	1.6364	2.9388

- GSI peaks at iteration 9 (0.8260), not the last iteration — so SOC tracks the best result, not just the final one.
- δ_m oscillates — expected behavior, not a bug.
- β_m and η change every iteration — the threshold is actively being pulled toward the optimal value, not sitting still.

OUR IMPLEMENTATION

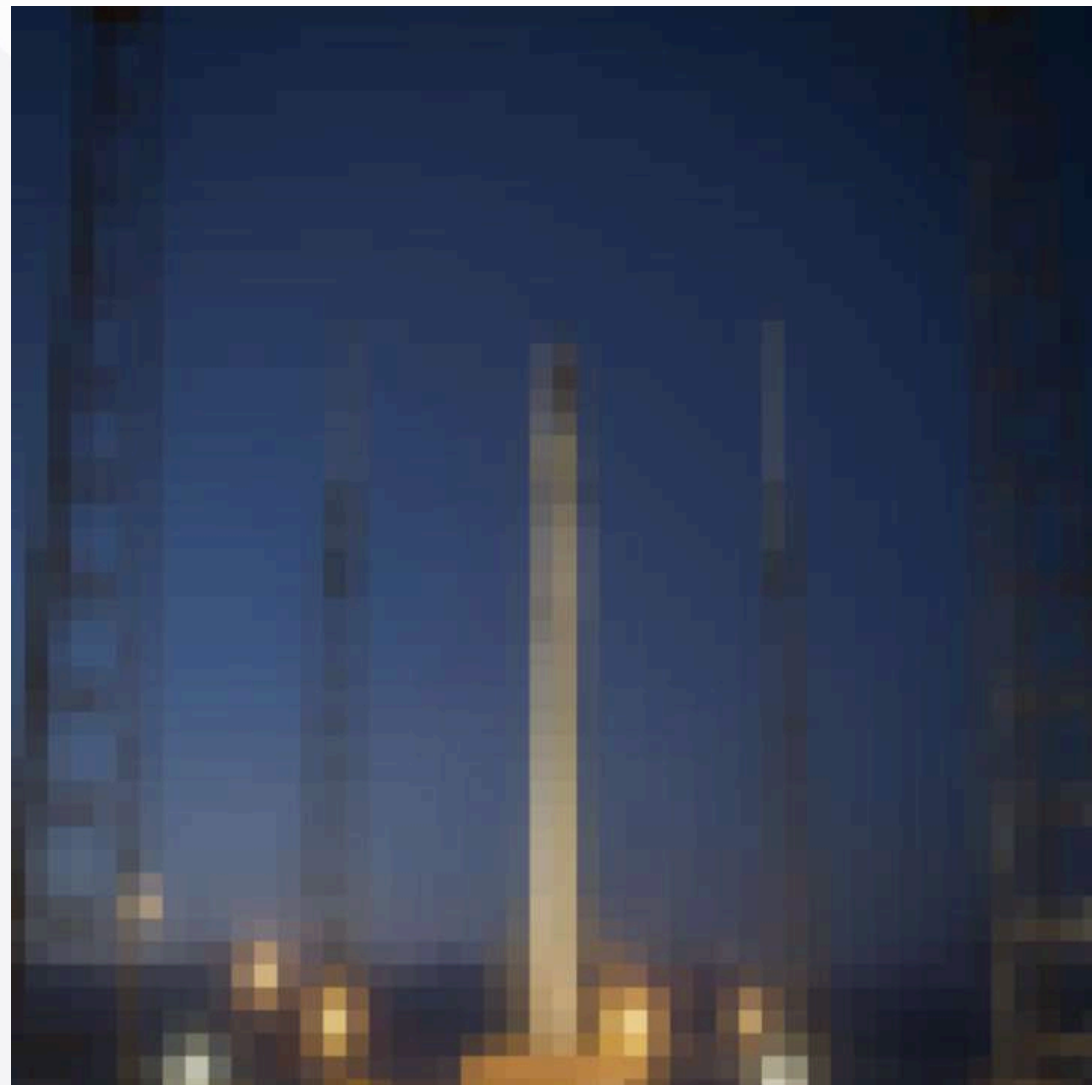
- Input data normalized into a unit hypercube so no feature dimension dominates distance calculations.
- Optimal k selected automatically by scanning $k = 2$ to 7 and picking the value that maximizes GSI — no manual specification needed.
- For each k , 10 β -optimization iterations are run: mountain potentials computed $\rightarrow$ highest-potential point selected as cluster center $\rightarrow$ nearby points assigned and removed $\rightarrow$ GSI evaluated via silhouette index.
- Lagrange polynomial fitted through the $(\delta m, S_m)$ pairs from each iteration $\rightarrow$ root η found where $St = 1 \rightarrow \beta m$ updated as $\eta/\delta m$, shifting the threshold toward the ideal silhouette value.
- Best GSI configuration across all 10 iterations retained — not just the final one, since convergence is conditional and the peak may occur mid-run.
- SOC compared against IMC-1 ($\beta = 1$ fixed), IMC-2 (heuristic β), and K-Means under identical experimental conditions, evaluated on all four validity indices: GSI, PI, SI, and DI.

FINDING OPTIMAL K

Unlike K-Means or FCM, SOC does not ask you to specify k. It finds it automatically

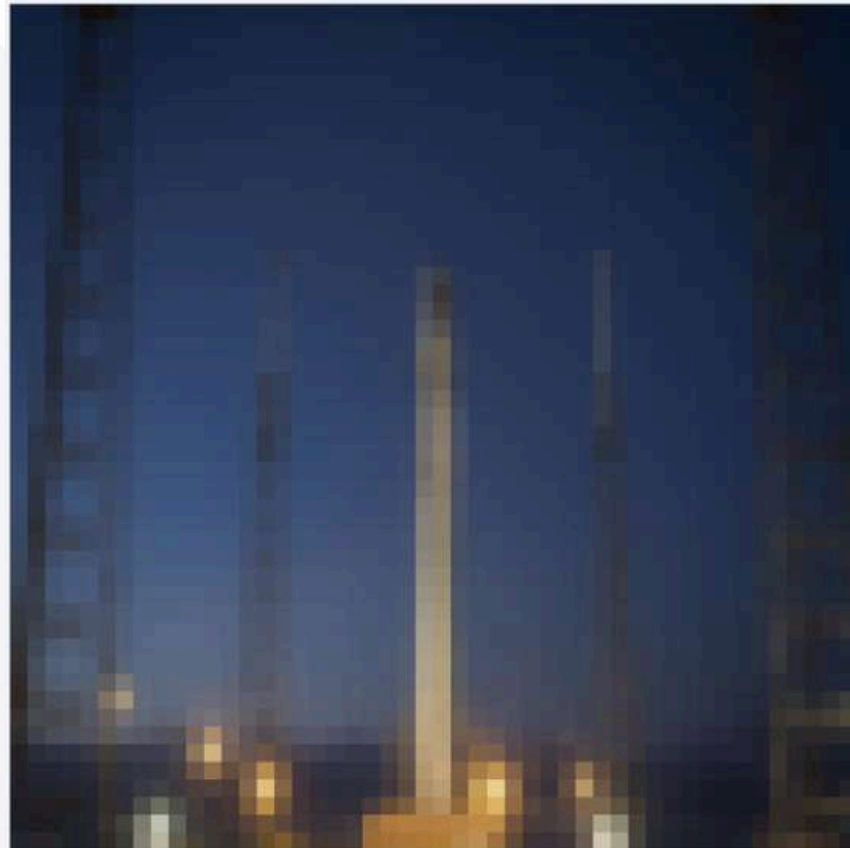
How it works:

- SOC runs clustering for $k = 2, 3, 4, 5, 6, 7$
- Records GSI for each value of k
- The k with the highest GSI is selected as optimal

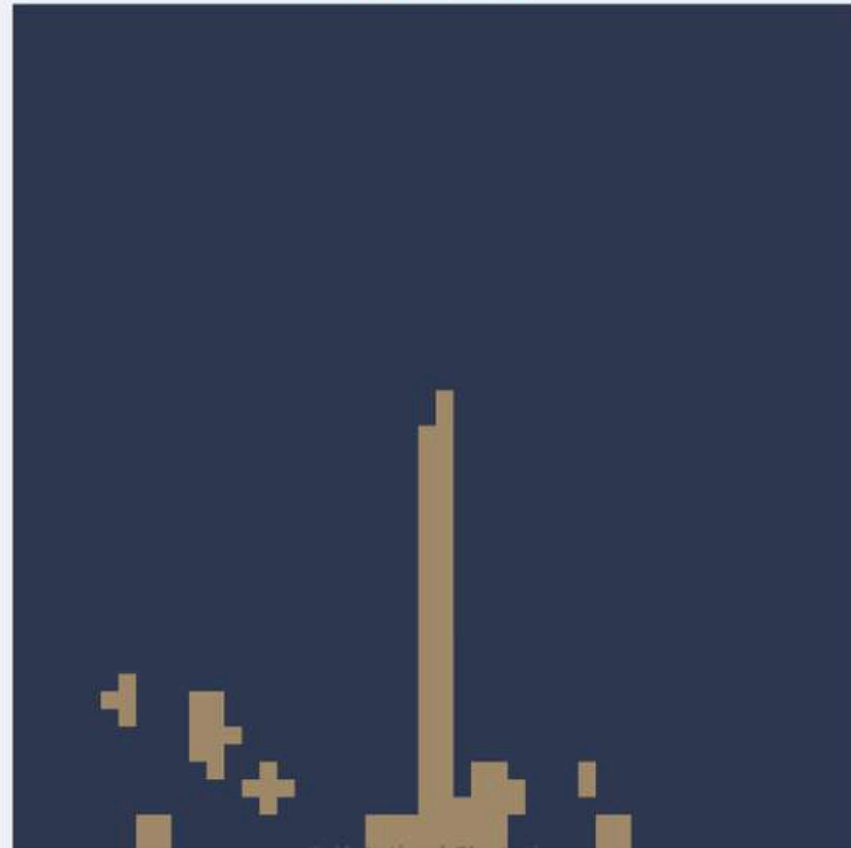


RESULTS - IMAGE SEGMENTATION

Original Image



SOC (k=2)



- SOC scanned $k=2$ through $k=7$ and automatically selected $k=2$ as optimal since it produced the highest GSI of 0.6222.
- At $k=2$, SOC outperformed all methods on every validity index — GSI of 0.6222, PI of 0.1514, SI of 0.0494, and DI of 1.0562, compared to the next best (IMC-2) scoring 0.4835, 0.4030, 0.1431, and 0.4225 respectively.
- Beyond numbers, SOC also correctly isolated the bright rocket tower as a distinct cluster — something all other methods failed to do — confirming that the optimized threshold delivers better results both numerically and visually.

RESULTS - SYNTHETIC DATASET

Three controlled Gaussian blob datasets were tested to verify SOC works beyond just images.

GSI COMPARISON:

Dataset	SOC	IMC-1	IMC-2	K-Means
2-Blob ($k = 2$)	0.7656	0.7633	0.7633	0.7656
3-Blob ($k = 3$)	0.6031	0.5980	0.4676	0.6031
4-Blob ($k = 4$)	0.7801	0.7801	0.6474	0.7801

Key observations:

- IMC-2 collapses on 3-blob and 4-blob — proving the heuristic factor is unreliable
- SOC never performs worse than the best alternative
- On simple well-separated data, SOC matches K-Means — confirming correctness

SOC VS OTHER METHODS

- SOC is the only method where every key feature is fully implemented — not assumed, not guessed.

Feature	K-Means	IMC-1	IMC-2	SOC
Handles Non-linearity	No	Yes	Yes	Yes
Adaptive Threshold	No	No	Partial	Yes
Uses Silhouette Feedback	No	No	No	Yes
Data-driven Optimization	No	No	No	Yes
Automatic k Detection	No	No	No	Yes
Consistent Cluster Quality	Medium	Medium	Varies	High

- K-Means and IMC-1 are rigid. IMC-2 tries to adapt but without any mathematical basis. SOC is the only method that uses actual cluster quality as feedback to improve itself — making it self-correcting by design.

PERKS OF SOC

- Smart threshold, not a guess — β_m is computed by solving a polynomial equation from the data. Not assumed, not heuristic.
- Each cluster gets its own threshold — Dense clusters get a tighter radius, sparse ones get a wider one. All other methods apply one threshold to everything.
- Self-correcting feedback loop — GSI is measured after every pass and used to improve the next iteration. Like getting test results and adjusting your preparation accordingly.
- Proven conditionally convergent — Not just trial and error. The threshold function is proven to converge using Weierstrass and Chebyshev theorems.
- Best-of-10 selection — SOC tracks the best configuration across all 10 iterations and returns that — not just the last result.



DRAWBACKS AND LIMITATIONS

- **$O(n^2)$ complexity**

Every point compared to every other point — real high-resolution images would be computationally infeasible.

- **Fixed 10 iterations**

Always runs all 10 even if converged early. Best GSI appeared at iteration 9 — last iteration added no value.

- **k range needs user input**

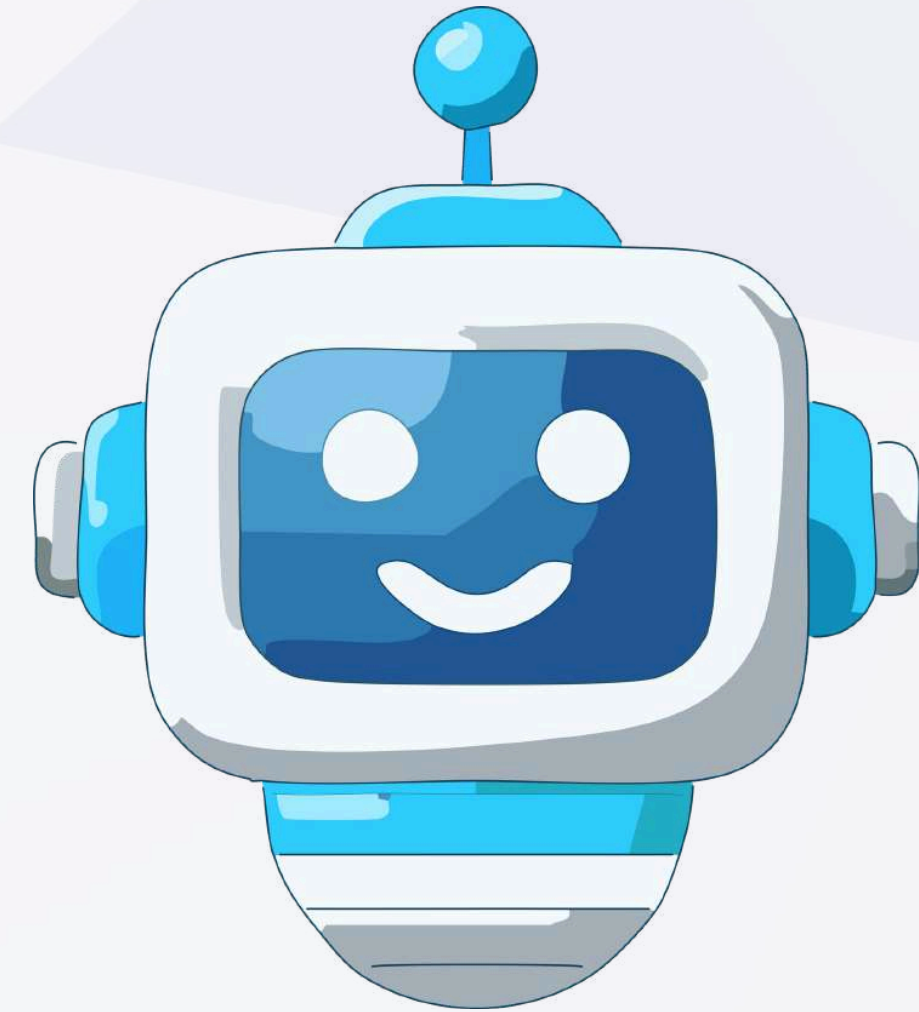
If true k lies outside the search range, SOC returns the wrong answer. Not fully automatic.

- **Lagrange instability**

When two δm values are very close, denominator approaches zero — causing η to spike. Observed in several iterations of our implementation.

- **RGB features only**

Two pixels with same color but different texture/location are treated as identical — limits real-world applicability.



FUTURE DIRECTIONS

Problem	Fix	Why it helps
$O(n^2)$ slowness	Approximate nearest-neighbor (e.g. KD-Tree, Ball Tree)	Instead of comparing every point to every other point, only search nearby region → reduces to $O(n \log n)$
Fixed 10 iterations	Adaptive stopping: halt when $ GSI_i - GSI_{i-1} < \epsilon$	No wasted iterations after convergence — also catches early convergence like iter 9 automatically
Lagrange instability	Replace with cubic splines	Splines fit smooth curves between points without oscillation — avoids η spiking when δm values are close
RGB only	Add HOG + LAB color space	HOG captures texture/edges, LAB separates brightness from color — gives richer features than raw RGB
Narrow comparison	Benchmark vs DBSCAN, FCM, EM, deep clustering	Current paper only compares IMC family — broader benchmark would establish where SOC truly stands



CONCLUSION

SOC was proposed to solve one specific problem — the threshold in IMC was never optimized properly. Everything else follows from fixing that.

What SOC changed:

The threshold function in IMC-1 and IMC-2 was either fixed or heuristically modified with no mathematical backing. SOC replaced this with a data-driven, iteratively optimized threshold computed via Lagrange interpolation — and that single change produced measurable improvements across every validity index on every tested dataset.

What the results show:

SOC outperformed IMC-1, IMC-2 and K-Means on all four validity indices — GSI, PI, SI and DI — on the rocket image at the automatically detected optimal $k=2$. On synthetic datasets, SOC matched the best method on simple data and outperformed all methods on overlapping data. IMC-2, despite being a more recent method than IMC-1, failed on 2 out of 3 synthetic datasets — confirming that heuristic modification without mathematical justification is not a reliable strategy

CONCLUSION

What we implemented:

Our Python implementation faithfully replicates all 15 steps of the SOC algorithm including the complete Lagrange β -optimization loop — which was the core missing piece in previous implementations. We tested on a real rocket image and three synthetic datasets, generating results consistent with the original paper.

Limitations we acknowledge:

SOC is not perfect. The $O(n^2)$ complexity limits scalability, the 10-iteration count is a heuristic choice, and the k search range still requires user input. These are open problems that future work can address through approximate nearest-neighbor methods, adaptive stopping criteria, and more stable interpolation techniques like cubic splines.

Final takeaway:

SOC proves that the right mathematical framework — even a relatively simple one like Lagrange interpolation — can outperform methods that rely purely on intuition or trial and error.

**THANK
YOU.**

